

Authentication and Authorization

- **Автентикация:** Процесът на проверка на самоличността на потребителя (логин/регистрация).
- **Оторизация:** Определяне на нивото на достъп на потребителя до различни части на приложението.

Ще добавим следните **роли** за управление на достъпа:

- **Гост:** Може само да разглежда каталога с книги.
- **Регистриран потребител:** Може да добавя и редактира книги.
- **Администратор:** Може да добавя, редактира и изтрива книги.

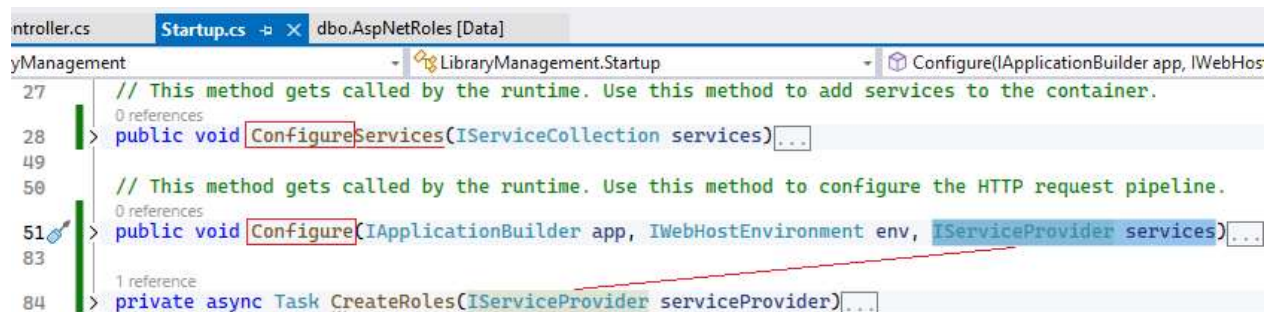
Забележка: Това не е най-оптималното решение, предвид по-новите версии .net 6.0 и нагоре, вкарване на Hard-core код за потребител с парола в нехеширан вид, това, че не е вкаран интерфейс за редакции с ролята във Views, това, че в таблицата UserRoles на БД, логнат потребител не се разпознава веднага като роля User, но този начин е достатъчно разбираем. Принципно е желателно да се направи отделен модел с отделен контролер и изглед специално за ролята и тяхното редактиране за дадените потребители да става графично.

В Startup.cs имаме 2 основни метода за настройки:

* ConfigureServices

* Configure

В случая, кодът е свит и те се виждат по-ясно. Под тях, ще добавим несинхронен метод за създаване на роли.



```
27 // This method gets called by the runtime. Use this method to add services to the container.
28 public void ConfigureServices(IServiceCollection services) {...}
49
50 // This method gets called by the runtime. Use this method to configure the HTTP request pipeline.
51 public void Configure(IApplicationBuilder app, IWebHostEnvironment env, IServiceProvider services) {...}
83
84 private async Task CreateRoles(IServiceProvider serviceProvider) {...}
```

Първоначално влизаме в ConfigureServices:

```
Create.cshtml Startup.cs X
LibraryManagement.Startup ConfigureServices(IServiceCollection services)
services.AddDefaultIdentity<IdentityUser>(options => options.SignIn.RequireConfirmedAccount = true)
    .AddRoles<IdentityRole>() добавяме възможност за роли
    .AddEntityFrameworkStores<ApplicationDbContext>();
services.AddControllersWithViews();
}
```

.AddRoles<IdentityRole>()

Кодът може да се персонализира по следния начин, колкото да демонстрираме кое какво прави:

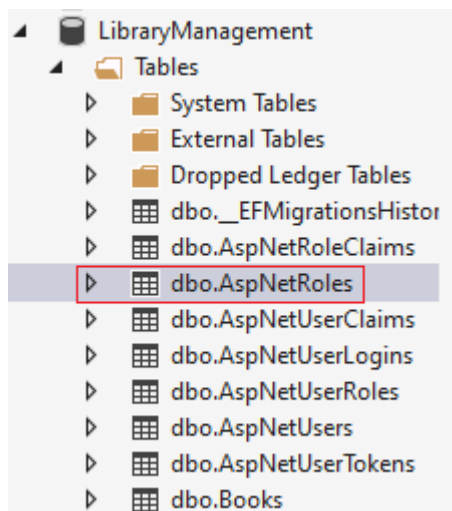
```
Startup.cs X BooksController.cs dbo.AspNetRoles [Data] dbo.AspNetUserRoles [Data]
LibraryManagement.Startup ConfigureServices(IServiceCollection services)
services.AddDefaultIdentity<IdentityUser>(options =>
{
    options.SignIn.RequireConfirmedAccount = false; // Не изисква потвърждение
    options.Password.RequireDigit = true; // Изисква поне една цифра
    options.Password.RequiredLength = 4; // Минимална дължина
    options.Password.RequireLowercase = false; // Не изисква малки букви
    options.Password.RequireUppercase = false; // Не изисква главни букви
    options.Password.RequireNonAlphanumeric = false; // Не изисква специални символи
})
    .AddRoles<IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>();
services.AddControllersWithViews();
services.AddRazorPages(); желателно
```

```
services.AddDefaultIdentity<IdentityUser>(options =>
{
    options.SignIn.RequireConfirmedAccount = false; //Без потвърждение
    options.Password.RequireDigit = true; // Изисква поне една цифра
    options.Password.RequiredLength = 4; // Минимална дължина
    options.Password.RequireLowercase = false; // Не изисква малки букви
    options.Password.RequireUppercase = false; //Не изисква главни букви
    options.Password.RequireNonAlphanumeric = false; //Без спец. символи
})
    .AddRoles<IdentityRole>()
```

Изпълнение на миграции

```
Add-Migration AddIdentity
Update-Database
```

Както знаем, миграциите са езикът, на който VisualStudio създава таблица на БД – след изпълнението им, забелязваме създаването на новата dbo.AspNetRoles



Най-добре е да свием кода на двата вградени метода за настройки ConfigureServices() и Configure(), за да запишем извън тях новия метод за създаване на роли:

```
Startup.cs  X  dbo.AspNetRoles [Data]
nt - LibraryManagement.Startup - Configure(IApplication
1 reference
private async Task CreateRoles(IServiceProvider serviceProvider)
{
    var roleManager = serviceProvider.GetRequiredService<RoleManager<IdentityRole>>();
    var userManager = serviceProvider.GetRequiredService<UserManager<IdentityUser>>();

    string[] roleNames = { "Admin", "User" };
    foreach (var roleName in roleNames)
    {
        if (!await roleManager.RoleExistsAsync(roleName))
        {
            await roleManager.CreateAsync(new IdentityRole(roleName));
        }
    }

    // Създаване на администраторски акаунт
    string adminEmail = "admin@gmail.com";
    string adminPassword = "admin123";

    var adminUser = await userManager.FindByEmailAsync(adminEmail);
    if (adminUser == null)
    {
        adminUser = new IdentityUser { UserName = adminEmail, Email = adminEmail };
        var result = await userManager.CreateAsync(adminUser, adminPassword);
        if (result.Succeeded)
        {
            await userManager.AddToRoleAsync(adminUser, "Admin");
        }
    }
}
```

Hard core seeding
с открита парола не е препоръчително!!!

Обръщаме внимание, че той е несинхронен!

```

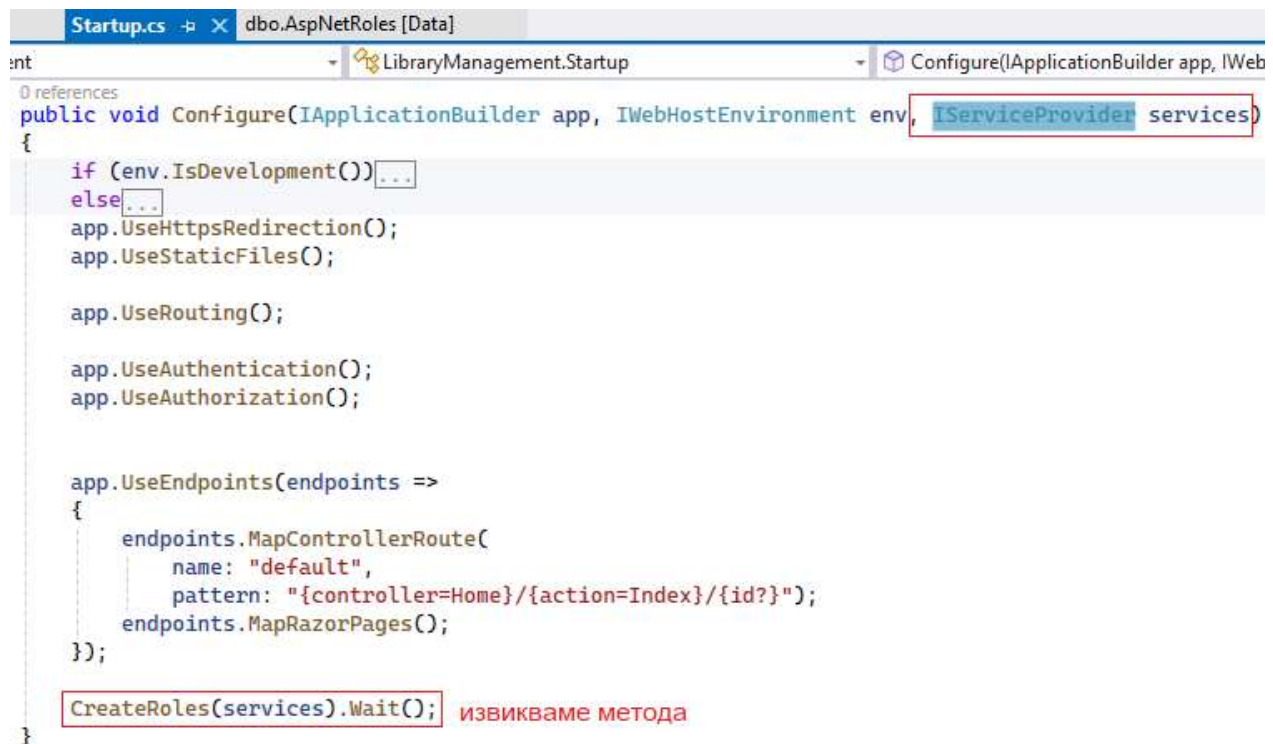
private async Task CreateRoles(IServiceProvider serviceProvider)
{
    var roleManager =
serviceProvider.GetRequiredService<RoleManager<IdentityRole>>();
    var userManager =
serviceProvider.GetRequiredService<UserManager<IdentityUser>>();

    string[] roleNames = { "Admin", "User" };
    foreach (var roleName in roleNames)
    {
        if (!await roleManager.RoleExistsAsync(roleName))
        {
            await roleManager.CreateAsync(new IdentityRole(roleName));
        }
    }
    // Създаване на администраторски акаунт
    string adminEmail = "admin@gmail.com";
    string adminPassword = "admin123";

    var adminUser = await userManager.FindByEmailAsync(adminEmail);
    if (adminUser == null)
    {
        adminUser = new IdentityUser { UserName = adminEmail, Email = adminEmail };
        var result = await userManager.CreateAsync(adminUser, adminPassword);
        if (result.Succeeded)
        {
            await userManager.AddToRoleAsync(adminUser, "Admin");
        }
    }
}

```

След като методът е създаден, можем да го извикаме там, където ни потрябва. Това е правилото за отделяне на повтарящ се код – логиката на метода е на едно място, а извикването му – (веднъж или няколко пъти, зависи кога ни потрябва) – на друго/и. Така кодът е по-лесен за структуриране и четене, логически е по-подреден.



```
Startup.cs  X  dbo.AspNetRoles [Data]
int  LibraryManagement.Startup  Configure(IApplicationBuilder app, IWeb
0 references
public void Configure(IApplicationBuilder app, IWebHostEnvironment env, IServiceProvider services)
{
    if (env.IsDevelopment()) ...
    else ...
    app.UseHttpsRedirection();
    app.UseStaticFiles();

    app.UseRouting();

    app.UseAuthentication();
    app.UseAuthorization();

    app.UseEndpoints(endpoints =>
    {
        endpoints.MapControllerRoute(
            name: "default",
            pattern: "{controller=Home}/{action=Index}/{id?}");
        endpoints.MapRazorPages();
    });

    CreateRoles(services).Wait(); извикваме метода
}
```

Дописваме услуга за променлива `services`, тип `IServiceProvider`, където начална главна буква `I` означава Interface.

`UseStaticFiles` – статични файлове като картинки и `.html`

`UseRouting` – пътят, по който адресира URL в брауъра

`UseAuthentication` и `UseAuthorization` – говорят сами за себе си

`UseEndpoints` – съответства на `Run` – последният слой (входна точка), в който е сложен пътят за зареждане по подразбиране `defaultRoute`

Последно – извикваме новия метод, който създадохме `CreateRoles(services).Wait();`

Остана да ограничим достъпа до определени действия в контролера и по този начин определим правата на ролите:

Задаваме глобален `[Authorize]`, чрез който забраняваме действията на всички нелогнати потребители наведнъж, но локално ще направим изключение за read-операцията, т.е. action GET – т.е. гостът само да може да разглежда каталога. За останалите – разрешаваме на `User` и `Admin` всички действия, като само `Admin` може да изтрива. За всеки метод се задават едни и същи позволени роли за Get + Post по двойки:

BooksController.cs* X dbo.AspNetUserRoles [Data] Startup.cs

LibraryManagement.Controllers.BooksCont Details(in

[Authorize]

глобален, за всички важи логване

1 reference

public class BooksController : Controller

{

private readonly ApplicationDbContext _context;

0 references

public BooksController(ApplicationDbContext context){...}

[AllowAnonymous]

// GET: Books

3 references

public async Task<IActionResult> Index(){...}

[Authorize(Roles = "Admin")]

// GET: Books/Details/5

0 references

public async Task<IActionResult> Details(int? id){...}

// GET: Books/Create само логнат - от глобалния Authorize

0 references

public IActionResult Create(){...}

[Authorize(Roles="Admin")]

Можем да тестваме с F5 всички действия за готовия админски акаунт, нелогнат гост и някакъв потребителски акаунт. Още след първото стартиране, таблицата се попълва:

ip.cs BooksController.cs dbo.AspNetRoles [Data] X

Max Rows: 1000

Id	Name	NormalizedNa...	ConcurrencySt...
490-86f21bae41e4	Admin	ADMIN	342c8e75-d3f2-...
d068d35b-3baf...	User	USER	f308e7c2-fe47-...
NULL	NULL	NULL	NULL

Ограничен достъп за нелогнат потребител – страница Details препраща към Login:

localhost:44386/Identity/Account/Login?ReturnUrl=%2FBooks%2FDetails%2F5

LibraryManagement Home Privacy Books Register Login

Log in

Use a local account to log in.

Email

Password

☐ Remember me?

Log in

[Forgot your password?](#)

Use another service to log in.

There are no external authentication services configured. See [this article about setting up this ASP.NET application to support logging in via external services.](#)

Ако потребителят се впише като обикновен user, пак ще откаже достъп, но с друг интерфейс:

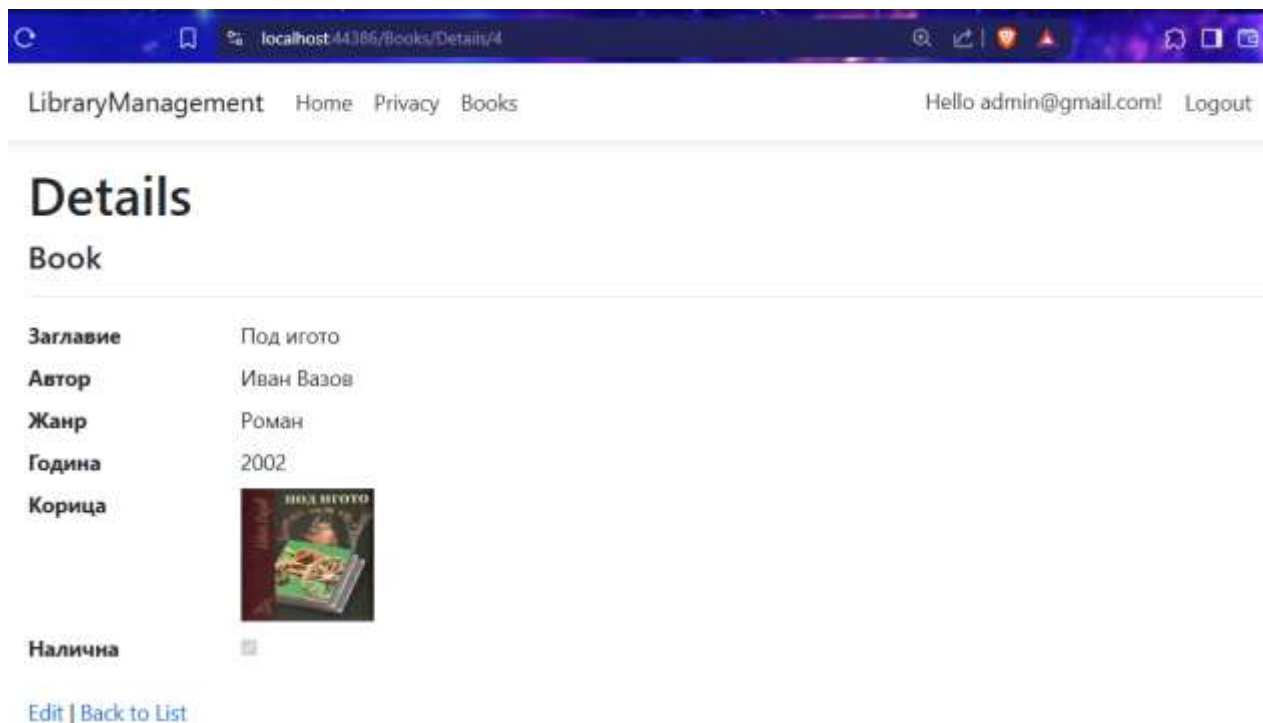
localhost:44386/Identity/Account/AccessDenied?ReturnUrl=%2FBooks%2FDetails%2F4

LibraryManagement Home Privacy Books Hello test1@test.bg! Logout

Access denied

You do not have access to this resource.

Разрешава се само на админ.



На всеки потребител User съответства id

Controller.cs			dbo.AspNetUsers [Data]	↗	✕
			Max Rows: 1000	⌵	🖨
Id	UserName	NormalizedUs...			
11689456-4a11-...	admin@gmail....	ADMIN@GMAI...			
529-fc084251ec5f	test1@test.bg	TEST1@TEST.BG			
aa6cb19f-7484-...	test@test.bg	TEST@TEST.BG			

На двете роли – Admin и User също съответстват уникални id

Controller.cs		dbo.AspNetUsers [Data]	dbo.AspNetRoles [Data]	↗	✕	dbo.AspNet
		Max Rows: 1000	⌵	🖨	🖨	
Id	Name	NormalizedNa...	ConcurrencyStamp			
2fddeb8f-a040-...	Admin	ADMIN	342c8e75-d3f2-4fce-a6cb-18b40f1a1017			
188-961d0c93826d	User	USER	f308e7c2-fe47-4da8-be6c-3fae5c492daf			
NULL	NULL	NULL	NULL			

Това означава, че ръчно можем да зададем на дадени имейли роля, като ръчно бръкнем в свързващата таблица, която е съставена от имената на другите 2 единични таблици (с първични ключове). Таблица UserRoles държи външните ключове към таблици Users + Roles:

dbo.AspNetUserRoles [Data] BooksC		
Max Rows: 1000		
	UserId	RoleId
	11689456-4a11-...	2fddeb8f-a040-...
	aa6cb19f-7484-...	d068d35b-3baf...
	5529-fc084251ec5f	NULL
*	NULL	NULL

По този начин, test@test.bg и test1@test.bg се разпознават като потребител User. Нека проверим:

```
BooksController.cs | dbo.AspNetUsers [Data] | dbo.AspNetRoles [Data]
LibraryManagement.Controllers.BooksCont | Details

[Authorize(Roles = "User, Admin")]
// GET: Books/Details/5
0 references
public async Task<IActionResult> Details(int? id) { ... }
```

Сега вече, тези имейли в роли User могат да получат достъп до Details

